

DBS302 - NoSQL Database Management

Practical 2C: Geo-Search Functionality with Redis Geospatial Indexes

**Student Name: Tandin Om
Student Number: 02230302**

1. Aim

Implement location-based search and distance queries using Redis geospatial indexes for storing and querying coordinates.

2. Objectives

- Explain how Redis stores geospatial data and what commands like GEOADD, GEOSEARCH, and GEODIST are used for
- Index points of interest (e.g., stores, drivers, or users) by longitude/latitude
- Implement "find nearest locations within a radius" queries
- Write a Python service that wraps these commands into high-level methods
- Finish exercises on bounding box search, geo-leaderboard, and top-N nearest locations

3. Conceptual Overview

The geospatial features in Redis are built on top of a data structure called a sorted set.

Key Commands:

- GEOADD key longitude latitude member - Add a location
- GEODIST key member1 member2 [unit] - Compute distance between two locations
- GEOSEARCH key FROMMEMBER member BYRADIUS radius unit - Search within radius from a member
- GEOSEARCH key FROMLONLAT lon lat BYRADIUS radius unit - Search within radius from coordinates
- GEOSEARCH key FROMLONLAT lon lat BYBOX width height unit -Search within a box

Analogy: Imagine a simple in-memory map service that has a point for each person's name, and the server can answer "who is near this point?" quickly.

4. CLI Demonstration - Adding Locations and Distance Calculation

```
○ (venv) macbookairm4chip@Tandin-Omu redis-practicals % redis-cli
127.0.0.1:6379> GEOADD geo:stores:thimphu 89.6390 27.4728 store_norzin
(integer) 1
127.0.0.1:6379> GEOADD geo:stores:thimphu 89.6530 27.4712 store_changzamtog
(integer) 1
127.0.0.1:6379> GEOADD geo:stores:thimphu 89.6490 27.4770 store_motithang
(integer) 1
127.0.0.1:6379> GEODIST geo:stores:thimphu store_norzin store_changzamtog km
"1.3931"
127.0.0.1:6379> |
```

The following commands were executed to add stores in Thimphu and calculate distances:

Command	Output	Purpose
GEOADD geo:stores:thimphu 89.6390 27.4728 store_norzin	(integer) 1	Add Norzin store
GEOADD geo:stores:thimphu 89.6530 27.4712 store_changzamtog	(integer) 1	Add Changzamtog store
GEOADD geo:stores:thimphu 89.6490 27.4770 store_motithang	(integer) 1	Add Motithang store
GEODIST geo:stores:thimphu store_norzin store_changzamtog km	"1.3931"	Distance between stores

Analysis: The distance between Norzin and Changzamtog is approximately 1.39 kilometers.

5. CLI Demonstration - Nearby Search (GEOSEARCH)

```
127.0.0.1:6379> GEOSEARCH geo:stores:thimphu FROMMEMBER store_norzin BYRADIUS 3 km WITHDIST WITHCOORD
1) 1) "store_norzin"
   2) "0.0000"
   3) 1) "89.6389988064766"
      2) "27.47279960831782"
2) 1) "store_changzamtog"
   2) "1.3931"
   3) 1) "89.65299993753433"
      2) "27.471200199266278"
3) 1) "store_motithang"
   2) "1.0917"
   3) 1) "89.64899808168411"
      2) "27.476999641278848"
127.0.0.1:6379> GEOSEARCH geo:stores:thimphu FROMLONLAT 89.6390 27.4728 BYRADIUS 3 km WITHDIST WITHCOORD
1) 1) "store_norzin"
   2) "0.0001"
   3) 1) "89.6389988064766"
      2) "27.47279960831782"
2) 1) "store_changzamtog"
   2) "1.3930"
   3) 1) "89.65299993753433"
      2) "27.471200199266278"
3) 1) "store_motithang"
   2) "1.0916"
   3) 1) "89.64899808168411"
      2) "27.476999641278848"
127.0.0.1:6379> █
```

Two methods were used to find stores within 3 km of Norzin:

Method 1: Using FROMMEMBER

text

```
GEOSEARCH geo:stores:thimphu FROMMEMBER store_norzin BYRADIUS 3 km WITHDIST WITHCOORD
```

Method 2: Using FROMLONLAT

text

```
GEOSEARCH geo:stores:thimphu FROMLONLAT 89.6390 27.4728 BYRADIUS 3 km WITHDIST WITHCOORD
```

Results:

Store	Distance (km)
store_norzin	0.0000
store_motithang	1.0916
store_changzamtog	1.3930

6. Exercise 1: Bounding Box Search (BYBOX)

```
(venv) macbookairm4chip@Tandin-Omu redis-practicals % redis-cli
127.0.0.1:6379> GEOADD geo:stores:thimphu 89.6390 27.4728 store_norzin
(integer) 0
127.0.0.1:6379> GEOADD geo:stores:thimphu 89.6530 27.4712 store_changzamtog
(integer) 0
127.0.0.1:6379> GEOADD geo:stores:thimphu 89.6490 27.4770 store_motithang
(integer) 0
127.0.0.1:6379> GEOADD geo:stores:thimphu 89.6410 27.4750 store_langjophakha
(integer) 0
127.0.0.1:6379> GEOADD geo:stores:thimphu 89.6360 27.4690 store_olakha
(integer) 0
127.0.0.1:6379> GEOSEARCH geo:stores:thimphu FROMLONLAT 89.6390 27.4728 BYBOX 3 3 km WITHDIST WITHCOORD
1) 1) "store_norzin"
   2) "0.0001"
   3) 1) "89.6389988064766"
      2) "27.47279960831782"
2) 1) "store_langjophakha"
   2) "0.3143"
   3) 1) "89.6409997344017"
      2) "27.474999746284126"
3) 1) "store_olakha"
   2) "0.5160"
   3) 1) "89.63600009679794"
      2) "27.469000061299973"
4) 1) "store_motithang"
   2) "1.0916"
   3) 1) "89.64899808168411"
      2) "27.476999641278848"
5) 1) "store_changzamtog"
   2) "1.3930"
   3) 1) "89.65299993753433"
      2) "27.471200199266278"
127.0.0.1:6379>
```

The BYBOX option allows searching within a rectangular area defined by width and height.

Results:

Store	Distance (km)
store_norzin	0.0001
store_langjophakha	0.3143
store_olakha	0.5160
store_motithang	1.0916
store_changzamtog	1.3930

Analysis: All 5 stores are within the 3km x 3km bounding box centered at Norzin.

7. Exercise 2: Geo-Search with Leaderboard Integration

```
127.0.0.1:6379> GEOADD geo:players:thimphu 89.6390 27.4728 alice
(integer) 1
127.0.0.1:6379> GEOADD geo:players:thimphu 89.6530 27.4712 bob
(integer) 1
127.0.0.1:6379> GEOADD geo:players:thimphu 89.6490 27.4770 charlie
(integer) 1
127.0.0.1:6379> GEOADD geo:players:thimphu 89.6410 27.4750 diana
(integer) 1
127.0.0.1:6379> GEOADD geo:players:thimphu 89.6360 27.4690 eve
(integer) 1
127.0.0.1:6379> GEOSEARCH geo:players:thimphu FROMLONLAT 89.6390 27.4728 BYRADIUS 2 km WITHDIST
1) 1) "alice"
   2) "0.0001"
2) 1) "diana"
   2) "0.3143"
3) 1) "eve"
   2) "0.5160"
4) 1) "charlie"
   2) "1.0916"
5) 1) "bob"
   2) "1.3930"
127.0.0.1:6379> ZSCORE leaderboard:game:season1 alice
(nil)
127.0.0.1:6379> ZSCORE leaderboard:game:season1 bob
(nil)
127.0.0.1:6379> ZSCORE leaderboard:game:season1 charlie
(nil)
127.0.0.1:6379> █
```

This exercise combines:

- Geospatial data - Player locations
- Sorted set leaderboard - Player scores

Results:

Player	Distance (km)
alice	0.0001
diana	0.3143
eve	0.5160
charlie	1.0916
bob	1.3930

Analysis: This demonstrates how to find nearby players and then retrieve their scores from a leaderboard, enabling features like "top players near you."

8. Exercise 3: Top N Closest Locations (COUNT Limit)

```
127.0.0.1:6379> GEOSEARCH geo:stores:thimphu FROMLONLAT 89.6390 27.4728 BYRADIUS 10 km WITHDIST WITHCOORD COUNT 3
1) 1) "store_norzin"
   2) "0.0001"
   3) 1) "89.6389988064766"
      2) "27.47279960831782"
2) 1) "store_langjophakha"
   2) "0.3143"
   3) 1) "89.6409997344017"
      2) "27.474999746284126"
3) 1) "store_olakha"
   2) "0.5160"
   3) 1) "89.63600009679794"
      2) "27.469000061299973"
127.0.0.1:6379> GEOSEARCH geo:stores:thimphu FROMLONLAT 89.6390 27.4728 BYRADIUS 10 km WITHDIST WITHCOORD COUNT 5
1) 1) "store_norzin"
   2) "0.0001"
   3) 1) "89.6389988064766"
      2) "27.47279960831782"
2) 1) "store_langjophakha"
   2) "0.3143"
   3) 1) "89.6409997344017"
      2) "27.474999746284126"
3) 1) "store_olakha"
   2) "0.5160"
   3) 1) "89.63600009679794"
      2) "27.469000061299973"
4) 1) "store_motithang"
   2) "1.0916"
   3) 1) "89.64899808168411"
      2) "27.476999641278848"
5) 1) "store_changzamtog"
   2) "1.3930"
   3) 1) "89.65299993753433"
      2) "27.471200199266278"
127.0.0.1:6379> 
```

The **COUNT** option limits the number of results returned, automatically sorted by distance (closest first).

Results:

#	Store	Distance (km)
---	-------	---------------

1	store_norzin	0.0001
2	store_langjophakha	0.3143
3	store_olakha	0.5160

Results:

#	Store	Distance (km)
1	store_norzin	0.0001
2	store_langjophakha	0.3143
3	store_olakha	0.5160
4	store_motithang	1.0916
5	store_changzamtog	1.3930

9. Python Geo-Search Service Implementation

```

(venv) macbookairm4chip@Tandin-Omu redis-practicals % python geo_search.py
=====
GEO-SEARCH SERVICE DEMO
=====

1. Adding stores to Thimphu...
Added: store_norzin (89.639, 27.4728)
Added: store_changzamtog (89.653, 27.4712)
Added: store_motithang (89.649, 27.477)
Added: store_langjophakha (89.641, 27.475)
Added: store_olakha (89.636, 27.469)
Added: store_babesa (89.662, 27.461)

2. Distance between stores:
Norzin → Changzamtog: 1.393 km

3. Stores within 2 km of Norzin (coordinates 89.6390, 27.4728):
store_norzin: 0.000 km
store_langjophakha: 0.314 km
store_olakha: 0.516 km
store_motithang: 1.092 km
store_changzamtog: 1.393 km

=====
EXERCISE 1: Bounding Box Search (BYBOX)
=====

Searching in a 3km x 3km box around Norzin:
store_norzin: distance 0.000 km
store_langjophakha: distance 0.314 km
store_olakha: distance 0.516 km
store_motithang: distance 1.092 km
store_changzamtog: distance 1.393 km
store_babesa: distance 2.622 km

=====
EXERCISE 2: Top N Nearest Locations
=====

Top 3 nearest stores from Norzin:
#1: store_norzin - 0.000 km
#2: store_langjophakha - 0.314 km
#3: store_olakha - 0.516 km

=====
EXERCISE 3: Distance Filter with Limit
=====

Stores within 3 km of Norzin (max 5 results):
store_norzin: 0.000 km
store_langjophakha: 0.314 km
store_olakha: 0.516 km
store_motithang: 1.092 km
store_changzamtog: 1.393 km
(venv) macbookairm4chip@Tandin-Omu redis-practicals %

```

A GeoSearchService class was implemented in Python with the following methods:

Method	Description	Redis Command
<code>add_location(name, lon, lat)</code>	Add a location	GEOADD
<code>distance_between(name1, name2)</code>	Calculate distance	GEODIST
<code>nearby(lon, lat, radius)</code>	Find locations within radius	GEOSEARCH

<code>search_by_box(lon, lat, width, height)</code>	Bounding box search	GEOSEARCH BYBOX
<code>get_nearest_n(lon, lat, n)</code>	Top N nearest locations	GEOSEARCH + sorting
<code>get_locations_within_distance(lon, lat, max_dist, limit)</code>	Distance filter with limit	GEOSEARCH + limit

10. Use Case Extension: Ride-Hailing Application

```
(venv) macbookairm4chip@Tandin-0mu redis-practicals % python ride_hailing.py
=====
RIDE-HAILING APP DEMO
=====

1. Adding drivers online...
Driver driver_001 updated to (89.639, 27.4728)
Driver driver_002 updated to (89.653, 27.4712)
Driver driver_003 updated to (89.649, 27.477)
Driver driver_004 updated to (89.641, 27.475)
Driver driver_005 updated to (89.636, 27.469)

2. Passenger requesting ride from Norzin (89.6390, 27.4728):

   Found 5 drivers within 2 km:
   #1: driver_001 - 0.000 km away
   #2: driver_004 - 0.314 km away
   #3: driver_005 - 0.516 km away
   #4: driver_003 - 1.092 km away
   #5: driver_002 - 1.393 km away

3. Simulating driver_004 moving closer to passenger...
Driver driver_004 updated to (89.64, 27.4735)

4. Updated nearby drivers:
   #1: driver_001 - 0.000 km away
   #2: driver_004 - 0.126 km away
   #3: driver_005 - 0.516 km away
   #4: driver_003 - 1.092 km away
   #5: driver_002 - 1.393 km away

5. Driver_002 goes offline...
Driver driver_002 removed

6. Final nearby drivers:
   #1: driver_001 - 0.000 km away
   #2: driver_004 - 0.126 km away
   #3: driver_005 - 0.516 km away
   #4: driver_003 - 1.092 km away
❖ (venv) macbookairm4chip@Tandin-0mu redis-practicals %
```

A ride-hailing service was implemented to demonstrate real-world application:

Features:

- Track driver locations in real-time
- Find nearby drivers for passenger requests
- Update driver locations as they move
- Remove drivers when they go offline

12. Key Learnings

Geospatial Commands:

- GEOADD stores coordinates with member names
- GEODIST calculates distance between locations
- GEOSEARCH with BYRADIUS finds locations within a circle
- GEOSEARCH with BYBOX finds locations within a rectangle
- COUNT limits results and returns them sorted by distance

Important Notes:

- Redis requires coordinates to be in the format of longitude, latitude
- Units can be: m (meters), km (kilometers), mi (miles), ft (feet)
- Geospatial indexes are built on top of sorted sets

Real-World Applications:

- Food delivery – find nearby restaurants
- Ride-hailing – find nearby drivers
- Social apps – find friends nearby
- Retail – find nearest store locations